PRODUCT API

What changed in today's GitHub pushes after 6:00 p.m.

A beginner-friendly guide to getProducts, getProduct, product view tracking, and the data flow behind them.

Scope: This guide covers the four commits on develop between 6:00 p.m. and 9:00 p.m. Korea time on Nov 5, 2026. The branch points to origin/develop at commit e538284.

The four pushes, in order

Time (KST)	Commit	What it added
18:32:20	cc93992	GET /product/all: query parsing, product filters, sorting and pagination.
19:10:26	e7862ea	GET /product/:id: one active product by ID, with optional member context.
19:33:15	008292b	View tracking foundation: ViewGroup, View schema, and ViewService.
20:31:28	e538284	Per-member view check, insert, and product view-count increment; fixes schema field casing.

How to use this guide: follow the request from the route, through the controller and service, into MongoDB, and back to the Postman response. Code terms are kept as written; each is explained in plain language.

1. Big picture: the request path

The code separates HTTP work from database work. The router chooses a controller, the controller reads the request and formats the response, and the service performs the MongoDB query. The View service records whether a signed-in member has viewed a product before.

Two product endpoints registered by the router

```
router.get("/product/all", productController.getProducts);
router.get(
  "/product/:id",
  memberController.retrieveAuth,
  productController.getProduct,
);
```

Part	Plain meaning	File
Router	Maps URL and HTTP method to the handler.	src/router.ts
Controller	Reads query/path values, calls service, sends HTTP status and JSON.	src/controllers/product.controller.ts
ProductService	Applies product rules and asks Mongoose/MongoDB for records.	src/models/Product.service.ts
ViewService	Finds or saves a member-product view record.	src/models/View.service.ts
Schemas and types	Describe saved fields and TypeScript shapes.	src/schema, src/libs/types

Flow for the product list

Example request

```
GET /product/all?order=createdAt&page=1&limit=3&productCollection=DISH&search=steak
```

- The router sends /product/all to productController.getProducts.
- The controller converts URL query text into numbers and a ProductInquiry object.
- ProductService builds the MongoDB match, sort, skip, and limit stages.
- MongoDB returns matching products; the controller sends them as JSON with HTTP 200.

Flow for one product

Example request

```
GET /product/65a856c038c6587e888b38c9
```

- retrieveAuth tries to find the member from the accessToken cookie. The middleware calls next() even when it catches an auth error, so this route can continue without a member; in that case no view is recorded.
- The controller reads the ID from req.params, then passes the ID and optional member ID to ProductService.
- The service finds a PROCESS product. For a signed-in member, it checks/saves a view and increments productViews once.

2. GET /product/all: controller and input type

Commit cc93992 added the SPA product-list endpoint and its ProductInquiry type. A query string is text at the HTTP boundary, so the controller converts it before handing it to the service.

Core controller code (formatting simplified; logic unchanged)

```
const { page, limit, order, productCollection, search } = req.query;
const inquiry: ProductInquiry = {
  order: String(order),
  page: Number(page),
  limit: Number(limit),
};
if (productCollection)
  inquiry.productCollection = productCollection as ProductCollection;
if (search) inquiry.search = String(search);
const result = await productService.getProducts(inquiry);
res.status(HttpCode.OK).json(result);
```

Code	What it does
req.query	Gets values after ? in the URL, such as page=1 and search=steak. These arrive as strings.
String(order)	Ensures order is represented as text.
Number(page), Number(limit)	Converts pagination values to numbers for MongoDB skip/limit.
if (productCollection)	Adds a category filter only when the caller supplied one.
as ProductCollection	Tells TypeScript to treat the text as one of the allowed collection names. It does not validate the runtime value.
if (search)	Adds a name search only when a non-empty search term exists.
getProducts(inquiry)	Passes all chosen filters to the database service.
status(OK).json(result)	Returns HTTP 200 and serializes the product array as JSON.

ProductInquiry: what the service is promised

src/libs/types/product.ts

```
export interface ProductInquiry {
  order: string;
  page: number;
  limit: number;
  productCollection?: ProductCollection;
  search?: string;
}
```

Required means the controller is expected to supply that field. The question mark means the caller may leave the optional filter out. The type helps TypeScript catch mismatches while coding; it does not automatically validate a Postman request.

How to call it: include order, page, and limit in the query string. Example: `/product/all?order=createdAt&page=1&limit=3`. Add productCollection=DISH or search=steak when you want those filters.

Error handling in the controller

The try block contains the normal path. If an operation throws, catch logs the failure. A known Errors instance uses its own HTTP code and JSON body; an unexpected error uses Errors.standard, normally a generic server error. This keeps internal failures from returning an arbitrary response shape.

3. GET /product/all: database pipeline

ProductService.getProducts turns the inquiry into an aggregation pipeline. Think of the pipeline as a small sequence of database steps: keep matching products, sort them, skip earlier pages, and take the requested number.

Core service logic

```
const match: T = { productStatus: ProductStatus.PROCESS };
if (inquiry.productCollection)
  match.productCollection = inquiry.productCollection;
if (inquiry.search) {
  match.productName = { $regex: new RegExp(inquiry.search, "i") };
}
const sort: T = inquiry.order === "ProductPrice"
  ? { [inquiry.order]: 1 }
  : { [inquiry.order]: -1 };
const result = await this.productModel.aggregate([
  { $match: match },
  { $sort: sort },
  { $skip: (inquiry.page * 1 - 1) * inquiry.limit },
  { $limit: inquiry.limit * 1 },
]).exec();
return result;
```

Step	Meaning
match starts with PROCESS	Only products whose productStatus is PROCESS are visible in this list. PAUSE and DELETE products are excluded.
productCollection filter	When supplied, restricts results to a category such as DISH or DRINK.
$regex with i	Searches productName without case sensitivity. “steak” can match “Steak”.
$match	Applies the filters before sorting or pagination.
$sort	Sorts by a database field. Value 1 means ascending; -1 means descending.
$skip	For page 1, skips 0. For page 2, skips one full page: (page - 1) * limit.
$limit	Caps the number of products in this page.
exec()	Runs the Mongoose aggregate query and resolves the result.

A real issue in the current sort condition

The code checks whether inquiry.order equals `"ProductPrice"` (capital P). The Product schema field is named `productPrice` (lowercase p). If the request sends the actual field name, the check is false and the code chooses descending order. This case mismatch makes the ascending-price branch hard to reach with the real field name.

Minimal correction

```
const sort: T = inquiry.order === "productPrice"
  ? { [inquiry.order]: 1 }
  : { [inquiry.order]: -1 };
```

This change makes productPrice sort ascending, while other requested fields continue to sort descending. It is the specific issue to correct in getProducts.

A useful note: aggregate() returns an array. An empty array is truthy in JavaScript, so `if (!result)` does not detect “no matching products.” Returning [] with HTTP 200 is normal for a list endpoint and makes the client responsible for showing an empty state.

4. GET /product/:id: controller and route

Commit e7862ea added the single-product endpoint. It uses the URL path for the product ID and passes any authenticated member ID along for view tracking.

Controller logic

```
const { id } = req.params;
const memberId = req.member?._id ?? null;
const result = await productService.getProduct(memberId, id);
res.status(HttpCode.OK).json(result);
```

Expression	Explanation
req.params.id	The part matched by :id in /product/:id.
req.member?._id	Reads the member ID if middleware attached a member. The ? safely handles a missing member.
?? null	Uses null when there is no member. The service then knows this is an anonymous view.
getProduct(memberId, id)	Delegates database lookup and view-count rules to the service.
HTTP 200 JSON	Returns the product object when found.

What retrieveAuth does in this route

The route passes through memberController.retrieveAuth before getProduct. That middleware reads the accessToken cookie, tries to set req.member, and then calls next(). In this project its catch block also calls next() without an error argument, so product retrieval continues even if authentication is absent or fails. The product is still retrievable; only member-specific view tracking is skipped.

Product lookup condition

```
const productId = shapeIntoMongooseObjectId(id);
const result = await this.productModel
  .findOne({ _id: productId, productStatus: ProductStatus.PROCESS })
  .exec();
if (!result) throw new Errors(HttpCode.NOT_FOUND, Message.NO_DATA_FOUND);
```

- shapeIntoMongooseObjectId converts the URL string into a MongoDB ObjectId value.
- findOne requires both the exact ID and PROCESS status to match.
- If the product is absent, has another status, or its ID differs, the service throws 404 No data is found!

The tutorial and local database have different records. Use an ID returned by your own GET /product/all request, and ensure that record has productStatus = PROCESS.

5. Product view tracking: when the count changes

The last two pushes add a small view history so one member does not increase a product's view counter every time they open it again. The authenticated member and the product ID together identify the view record.

Logic inside getProduct

```
if (memberId) {
  const input: ViewInput = {
    memberId: memberId,
    viewRefId: productId,
    viewGroup: ViewGroup.PRODUCT,
  };
  const existView = await this.viewService.checkViewExistance(input);
  if (!existView) {
    await this.viewService.insertMemberView(input);
    result = await this.productModel.findByIdAndUpdate(
      productId,
      { $inc: { productViews: +1 } },
      { new: true },
    ).exec();
  }
}
return result;
```

Situation	Behavior
Anonymous request	memberId is null. Product is returned; no View record is written and productViews is not increased.
Member opens this product for first time	No matching View exists. Save the view, then atomically add 1 to productViews.
Same member opens again	A matching View exists. Skip insert and skip the increment.
Product not PROCESS or ID not found	Throw 404 before view tracking begins.

Why $inc and { new: true } are used

- `$inc` tells MongoDB to increase a number in the database, instead of reading a value and writing back an old value. This avoids a simple lost-update problem when multiple requests arrive close together.
- `{ new: true }` asks Mongoose to return the updated product document. That refreshed record becomes the response.
- `let result` is used because the initial found product is replaced by the updated product after the increment.

The increment happens after the View record is inserted. If view insertion fails, the increment does not happen. If the product update fails afterward, the View record may already exist; the next request can then see the record and skip incrementing. This is a possible partial-failure edge case to know about.

6. View schema and service, including the casing fix

Commit 008292b introduced the View model and service skeleton. Commit e538284 added the typed input, existence check, insert operation, and the getProduct integration. That last commit also corrected two schema field names.

View schema fields

```
const viewSchema = new Schema(
  {
    viewGroup: { type: String, enum: ViewGroup, required: true },
    memberId: { type: Schema.Types.ObjectId, required: true, ref: "Member" },
    viewRefId: { type: Schema.Types.ObjectId, required: true },
  },
  { timestamps: true },
);
```

Field	Purpose
viewGroup	Which kind of item was viewed. Current enum contains PRODUCT.
memberId	The member who viewed it; stored as an ObjectId reference to Member.
viewRefId	The ID of the viewed product.
timestamps: true	Mongoose adds createdAt and updatedAt automatically.

The exact schema bug and correction

Before the fix, the schema declared `ViewGroup` and `viewrefId`. The service and TypeScript interface send `viewGroup` and `viewRefId`. Mongo field names are case-sensitive. Mongoose therefore did not see the expected required fields under the declared names, and saving could fail with a validation error. The catch mapped that lower-level error to the user-facing message “Create is failed!”.

ViewService operations

```
findOne({ memberId: input.memberId, viewRefId: input.viewRefId })

return await this.viewModel.create(input);
```

- checkViewExistance searches for a view row with the same memberId and viewRefId.
- insertMemberView writes the ViewInput document. On failure it logs the original error and throws a 400 Errors object with Message.CREATE_FAILED.
- ViewInput requires memberId, viewRefId, and viewGroup. The same spellings must be used in the interface, service, and schema.

The corrected schema names are viewGroup and viewRefId. This aligns the database model with the code that constructs and queries the view record.

One implementation detail: checkViewExistance currently checks memberId and viewRefId, but not viewGroup. Since the enum currently only has PRODUCT, this is enough for this feature. If other view groups are added later, include viewGroup in the lookup so an ID collision across different item types cannot be mistaken for the same view.

7. Practical Postman checklist and explanation script

Test the product list

- Method: GET. URL: http://localhost:3003/product/all?order=createdAt&page;=1&limit;=3
- Optional filter: add &productCollection;=DISH. Search: add &search;=steak.
- Expect HTTP 200 with a JSON array. It may be [] if nothing matches all filters.

Test one product and its view count

- First call GET /product/all and copy an _id from your own database. Confirm productStatus is PROCESS.
- Call GET /product/{that-id} with no accessToken cookie. Expect the product; view count should not be member-tracked.
- Log in so Postman has the accessToken cookie. Call the same product endpoint. For the first view by this member, a View row is saved and productViews increases by one.
- Call it again as the same member. The view already exists, so productViews should stay the same.
- If a signed-in request returns “Create is failed!”, inspect the terminal for the logged ViewModel validation/database error. Confirm View schema uses viewGroup, memberId, and viewRefId.

A useful explanation you can say to your instructor

“The list endpoint reads filters from the query string. The controller converts the page and limit to numbers and packages the input as ProductInquiry. The service filters to PROCESS products, optionally filters by category and name, sorts, then applies skip and limit. The single-product endpoint reads the ID from the path and checks that the product is PROCESS. If a member is known, it checks whether that member already has a View row for the product. It inserts one and increments productViews only for the first view. The schema field names must match the input names exactly.”

Main terms in plain language

Term	Simple meaning
Controller	The HTTP desk: receives request details and sends a response.
Service	The business logic and database work.
Schema	Rules describing how MongoDB documents are shaped.
Interface / type	A TypeScript description of the fields code expects.
Aggregation	A chain of database operations such as match, sort, skip, and limit.
ObjectId	MongoDB’s ID format; URL text is converted before querying.
Middleware	A function that runs between router matching and the controller.

Source trail: commits cc93992, e7862ea, 008292b, e538284 on develop/origin/develop. Main files: src/router.ts; src/controllers/product.controller.ts; src/models/Product.service.ts; src/models/View.service.ts; src/schema/View.model.ts; src/libs/types/product.ts; src/libs/types/view.ts; src/libs/enums/product.enum.ts; src/libs/enums/view.enum.ts.

Correction note: product price sorting

This note clarifies the getProducts sort issue described on page 4. The Product schema field is spelled `productPrice` (lowercase p), while the service checks for `ProductPrice` (uppercase P). MongoDB field names are case-sensitive.

Why both input spellings matter

Request order value	What the current code does	Problem
productPrice	Does not equal ProductPrice, so chooses descending (-1).	Ascending price request gets the wrong direction.
ProductPrice	Uses ProductPrice as the Mongo sort field.	That field is not in the schema, so sorting by price will not work as intended.

Use the real MongoDB field name for the sort key, and treat either spelling as a request for ascending price:

```
const sortField =
  inquiry.order === "ProductPrice" ? "productPrice" : inquiry.order;

const sortDirection = sortField === "productPrice" ? 1 : -1;
const sort: T = { [sortField]: sortDirection };
```

This supports a client that sends the tutorial-style alias ProductPrice and a client that sends the schema field productPrice. Both resolve to the actual database key productPrice and sort from low to high. Other allowed order fields remain descending under this rule.

If you change this in the code, update the current getProducts sort block in src/models/Product.service.ts. The existing code has not been edited for this issue yet.